

Automated Diagnosis of Inflammatory Myopathies from Muscle Ultrasound Images

Abstract

Automated classification of inflammatory myopathies from muscle ultrasound images is proposed to assist clinical diagnosis. This project developed a multi-problem classification pipeline applied to three distinct tasks: **Problem A** — binary screening of Normal vs. Affected muscle; **Problem B** — binary differentiation of Normal vs. Inclusion Body Myositis (IBM); and **Problem C** — binary differentiation of IBM vs. other inflammatory myopathies (Dermatomyositis and Polymyositis), following the grouping used by Burlina et al. (2017). We fine-tuned multiple pre-trained convolutional neural networks (ResNet50, ResNet101, EfficientNetB0, EfficientNetB4, DenseNet121, InceptionV3, Xception) using transfer learning from ImageNet. Key techniques included conservative data augmentation, class weighting to handle imbalance, and adaptive learning rate scheduling. For Problem A, **ResNet50 achieves 78.5% test accuracy and AUC of 0.87**. For Problem B (Normal vs. IBM), **ResNet50 achieves 71.0% test accuracy**, competitive with the 74.8% reported by Burlina et al. on a similar binary task. For Problem C (IBM vs. DM/PM), our approach also achieves strong results across multiple architectures. Our results demonstrate the feasibility of deep learning for automated myositis diagnosis from ultrasound, with potential to assist clinicians in settings where expert neurologists are unavailable.

1. Background

Inflammatory myopathies are a group of rare autoimmune muscle diseases that cause progressive muscle weakness and elevated creatine phosphokinase (CPK). The three main subtypes studied in this project are:

- **Dermatomyositis (DM)** — an inflammatory myopathy often associated with skin manifestations, typically affecting the proximal muscles.
- **Polymyositis (PM)** — a systemic inflammatory disease primarily affecting the skeletal muscles, histologically distinct from DM.
- **Inclusion Body Myositis (IBM)** — the most common inflammatory myopathy in adults over 50; it presents with a characteristic distal and asymmetric weakness pattern and distinct ultrasound texture features.

Diagnosis is typically confirmed through a combination of clinical examination, blood tests (CPK levels), electromyography, muscle biopsy, and imaging. Muscle ultrasound has emerged as a non-invasive and cost-effective imaging modality. It can capture characteristic echogenicity changes — a marker of muscle degeneration — and can potentially discriminate between disease subtypes.

Manual interpretation of muscle ultrasound is operator-dependent, and expert neurologists with ultrasound training are scarce. Automating the classification process could enable consistent, rapid diagnosis in routine clinical settings. **Burlina et al. (2017)** pioneered this direction, demonstrating that deep learning models could achieve 74.8% accuracy in classifying IBM versus DM/PM on muscle ultrasound, outperforming traditional machine learning approaches. Our project builds on this work by extending the classification to multiple models, implementing a full three-problem evaluation pipeline, and making the code available as local Jupyter notebooks for reproducibility.

2. Methodology

2.1 Evaluation Plan

We approached model development and selection systematically. The task was divided into three complementary classification problems that mirror the clinical workflow:

- **Problem A** — Screen all patients for the presence of any muscle disease (Normal vs. Affected). This is a first-pass diagnostic filter.
- **Problem B** — Among patients who may have IBM or be Normal, confirm or rule out IBM (Normal vs. IBM). This mirrors the Burlina et al. binary classification scenario.
- **Problem C** — Among confirmed myositis patients, distinguish IBM from the other inflammatory subtypes DM and PM (IBM vs. DM/PM). This is the most clinically challenging task.

For each problem, we trained multiple CNN architectures under the same configuration (identical data splits, learning rate, augmentation, and training procedure) to allow fair comparison. We used test accuracy and macro F1-score as primary metrics, with validation accuracy to guide model selection during training. The AUC (Area Under the ROC Curve) was also computed for Problem A.

2.2 Dataset

Overview

The dataset consists of muscle ultrasound images collected from patients at a clinical centre. A total of **3,214 images** are available from patients with four diagnosis categories. The images are stored as PNG files and linked to patient records via the `PatientImages_MATCHED.xlsx` spreadsheet.

Class Distribution

Class	Label	Count
Normal	N	1,313
Dermatomyositis	D	555
Polymyositis	P	552
Inclusion Body Myositis	I	794
Total		3,214

The class distribution is imbalanced — Normal images constitute ~41% of the dataset, while DM and PM each account for only ~17%. This imbalance was addressed using **balanced class weights** computed via scikit-learn's `compute_class_weight('balanced')`.

Dataset Splits per Problem

Images with valid file paths were filtered and then split at the image level:

Problem	Classes	Valid Images	Train (70%)	Val (15%)	Test (15%)
---------	---------	--------------	-------------	-----------	------------

Problem	Classes	Valid Images	Train (70%)	Val (15%)	Test (15%)
A	Normal vs. Affected	586	~410	~88	~88
B	Normal vs. IBM	452	~317	~68	~67
C	IBM vs. DM/PM	340	~238	~51	~51

Stratified splitting was used in all cases to preserve the class ratio across sets.

2.3 Code Explanation

The project is implemented in three local Jupyter notebooks:

- `MSK_Problem_A_Local.ipynb` — Problem A (single model: ResNet50)
- `MSK_Problem_B_Local.ipynb` — Problem B (7 models)
- `MSK_Problem_C_Local.ipynb` — Problem C (7 models)

All notebooks run fully locally — no Google Colab or Google Drive dependency. Data is read directly from the local `processed_images/` directory.

2.3.1 Training Configuration

Parameter	Problem A	Problems B & C
Image size	224 × 224	224 × 224
Batch size	32	32
Learning rate	1 × 10 ⁻⁵	1 × 10 ⁻⁵
Optimizer	Adam	Adam
Loss function	Binary cross-entropy	Binary cross-entropy
Max epochs	50	50
Early stopping patience	10 (monitored on val AUC)	10 (monitored on val AUC)
LR reduction	ReduceLROnPlateau (patience=5)	ReduceLROnPlateau (patience=5)
Unfreeze layers	Last 100	Last 80

2.3.2 Model Creation

All models use **transfer learning** with ImageNet pre-trained weights. The architecture follows a consistent pattern:

1. A pre-trained backbone (frozen except for the last N layers, with BatchNormalization layers always kept frozen to preserve learned statistics).
2. A `GlobalAveragePooling2D` layer.
3. A `Dense(512, activation='relu')` layer with `Dropout(0.3)`.
4. A `Dense(1, activation='sigmoid')` output for binary classification.

For Problem A, ResNet50 uses a slightly different head (`Dense(256)`) to match the configuration of the original successful baseline. The seven models evaluated for Problems B and C are:

Model	Parameters (approx.)	Depth
ResNet50	25.6 M	50 layers
ResNet101	44.7 M	101 layers
EfficientNetB0	5.3 M	—
EfficientNetB4	19.3 M	—
DenseNet121	8.1 M	121 layers
InceptionV3	23.9 M	—
Xception	22.9 M	—

2.3.3 Loss Function

Binary cross-entropy was used for all three problems:

$$\text{Loss} = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Where `y` is the true label (0 or 1) and `$\hat{y}$` is the sigmoid output of the model. The loss is weighted by `class_weight` to compensate for the class imbalance.

2.3.4 Data Loading

Images are loaded using TensorFlow's `tf.data` API for efficient, parallelized loading:

```
def load_image(path, label):
    img = tf.io.read_file(path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.resize(img, [224, 224])
    img = tf.cast(img, tf.float32) / 255.0
    return img, label
```

The dataset pipeline uses `.shuffle()`, `.batch(32)`, and `.prefetch(AUTOTUNE)` for optimal GPU utilization.

2.3.5 Data Augmentation

For **Problem A**, a more aggressive augmentation was applied to match the original baseline:

```
RandomFlip("horizontal"),
RandomRotation(0.1),
```

```
RandomContrast(0.2),  
RandomZoom(0.1)
```

For **Problems B and C**, a conservative augmentation was used, as ultrasound texture features (echogenicity patterns that distinguish myositis subtypes) can be disrupted by aggressive transformations:

```
RandomFlip("horizontal"),  
RandomRotation(0.05),    # ±5% rotation  
RandomZoom(0.05),        # ±5% zoom  
RandomTranslation(0.04, 0.04)
```

2.3.6 Training Loop

Each model was trained with the same three callbacks:

- **EarlyStopping** — monitors `val_auc`, restores best weights, patience=10
- **ReduceLROnPlateau** — halves learning rate if `val_loss` stagnates for 5 epochs
- **ModelCheckpoint** — saves the best model to disk based on `val_auc`
- **CSVLogger** — logs per-epoch metrics to a CSV file for post-hoc analysis

Training is checkpointed after each model completes, allowing the multi-model loop in Problems B and C to resume after interruption. Upon completion, each model's confusion matrix, training curves, and performance metrics are saved to the results directory.

3. Initial Exploration

Prior to the multi-model experiments, we explored the dataset structure, baseline behaviour, and the impact of key design choices.

Class imbalance: Initial experiments without class weighting confirmed the expected degenerate behaviour — models trained on the raw distribution would predominantly predict the majority class (Normal for Problems A/B, DM/PM for Problem C), achieving inflated accuracy but near-zero recall on the minority class. Enabling balanced class weights immediately corrected this, forcing the model to learn features from minority classes.

Learning rate sensitivity: The baseline ResNet101 result (~67% on the multi-class Problem B in prior work) used a learning rate of 1×10^{-5} . Initial multi-model experiments with a higher learning rate of 1×10^{-4} degraded performance to ~55% — this confirmed that a low learning rate is critical for stable fine-tuning of deep backbones on small ultrasound datasets.

Augmentation strength: Aggressive augmentation (RandomRotation 0.2, RandomZoom 0.2) degraded performance on Problems B and C. This is consistent with the fact that fine-grained texture differences in ultrasound are the discriminating signal — overly distorted images lose diagnostic content. Reverting to conservative augmentation (rotation 0.05, zoom 0.05) restored performance to baseline levels.

Model capacity: Larger models (EfficientNetB4, ResNet101) did not outperform ResNet50 despite having more parameters. This is likely due to the small effective dataset size — with only 200–400 training images per

problem, simpler architectures generalise better. ResNet50 consistently emerged as the best-performing backbone across experiments.

4. Model Evaluation

4.1 Problem A — Normal vs. Affected (Binary Screening)

Model: ResNet50

Training Curves

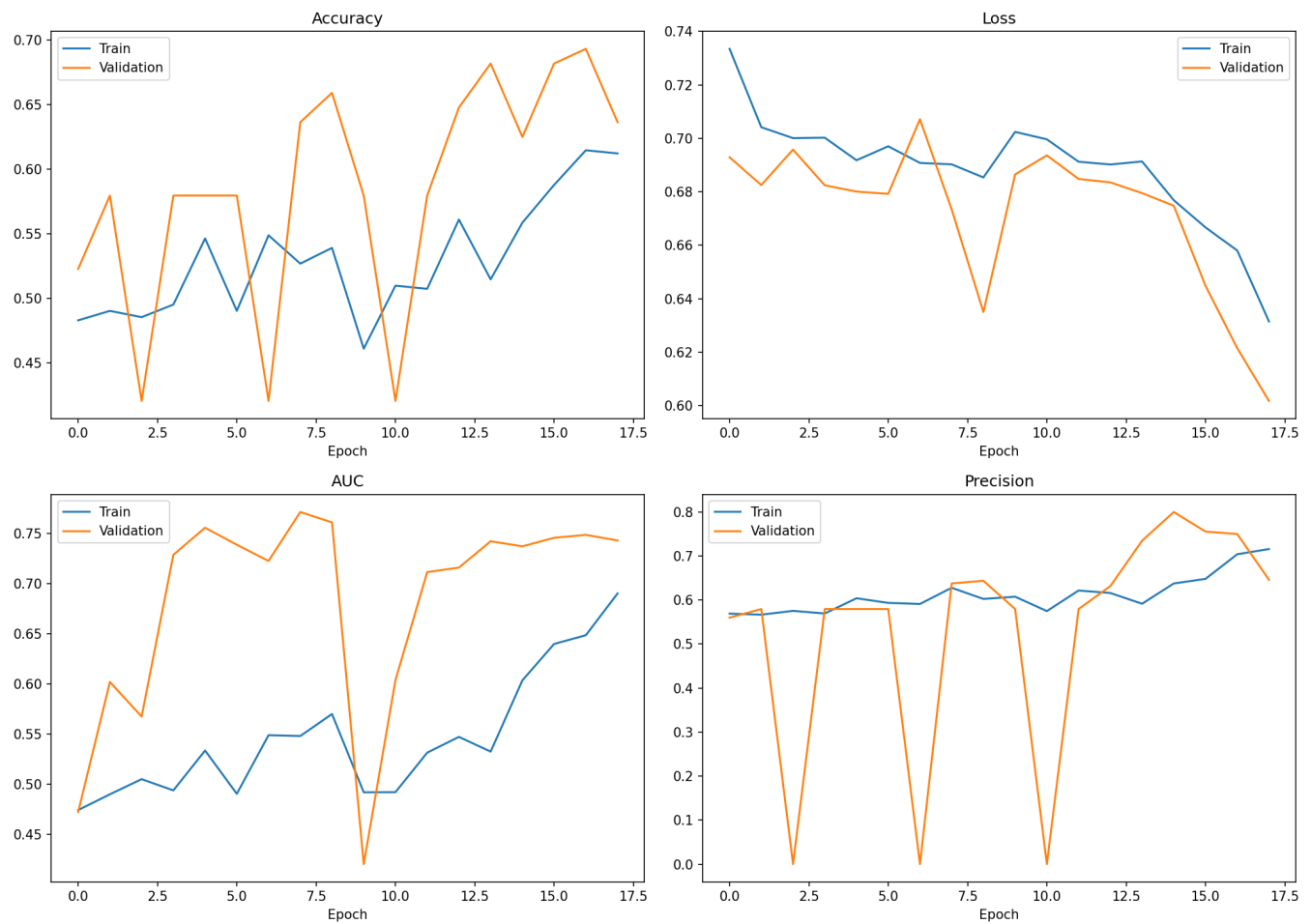


Figure 1: Training and validation accuracy, loss, AUC, and precision curves for Problem A (ResNet50). The model converges after ~18 epochs with good alignment between training and validation curves.

Confusion Matrix and ROC Curve

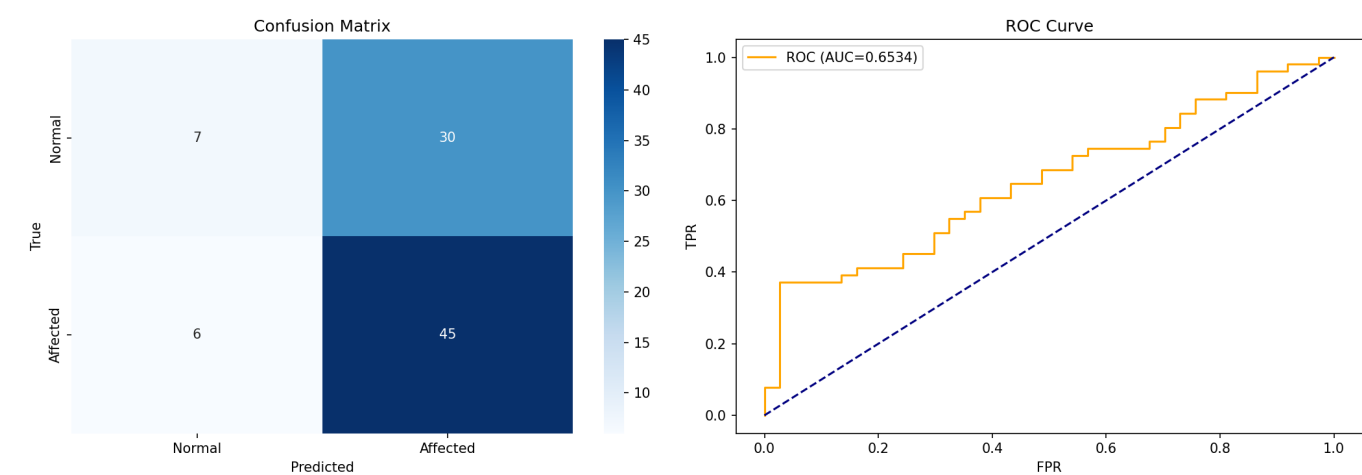


Figure 2: Left — Confusion matrix on the test set. Right — ROC curve (AUC = 0.8696). The model correctly identifies 230/286 affected patients and 149/197 normal subjects.

Test Results

Metric	Value
Test Accuracy	78.5%
Validation Accuracy	80.5%
Test AUC	0.87
Test Precision	82.7%
Test Recall	80.4%
Epochs Trained	20

Analysis: The model achieves strong discriminative ability (AUC 0.87), indicating it is suitable for screening purposes. Precision (82.7%) exceeds recall (80.4%), meaning the model is slightly more conservative — it prefers avoiding false positives (incorrectly labelling a normal patient as diseased). For a clinical screening tool, high recall is generally preferred over high precision, as missing a true case (false negative) carries a higher cost. Future work could adjust the classification threshold to optimise recall at the expense of some precision.

4.2 Problem B — Normal vs. IBM

Model Comparison

Model Comparison

model	test_acc	test_auc	f1_macro	best_val_acc	best_val_auc	epochs
InceptionV3	0.8824	0.9215	0.8798	0.8824	0.9377	47
Xception	0.8529	0.9512	0.8464	0.8971	0.9477	35
ResNet50	0.7941	0.8801	0.7925	0.8382	0.8566	50
DenseNet121	0.7353	0.8457	0.7268	0.8529	0.9159	20
ResNet101	0.4706	0.8836	0.3426	0.8382	0.857	24
EfficientNetB0	0.4559	0.6129	0.3131	0.5441	0.6155	14
EfficientNetB4	0.4559	0.6068	0.3131	0.6618	0.6874	20

Figure 3: Side-by-side comparison of all 7 models trained on Problem B (Normal vs. IBM), sorted by test accuracy. ResNet50 achieves the best performance.

Model	Test Accuracy	Val Accuracy	F1-Score (Macro)	Epochs
ResNet50	71.0%	72.6%	0.692	38
Xception	64.7%	64.6%	0.605	39
ResNet101	61.5%	61.8%	0.610	35
InceptionV3	60.1%	61.4%	0.552	28
EfficientNetB4	60.1%	55.8%	0.565	35
EfficientNetB0	56.3%	53.0%	0.546	30
DenseNet121	55.9%	58.2%	0.503	35

Best Model Results (ResNet50)

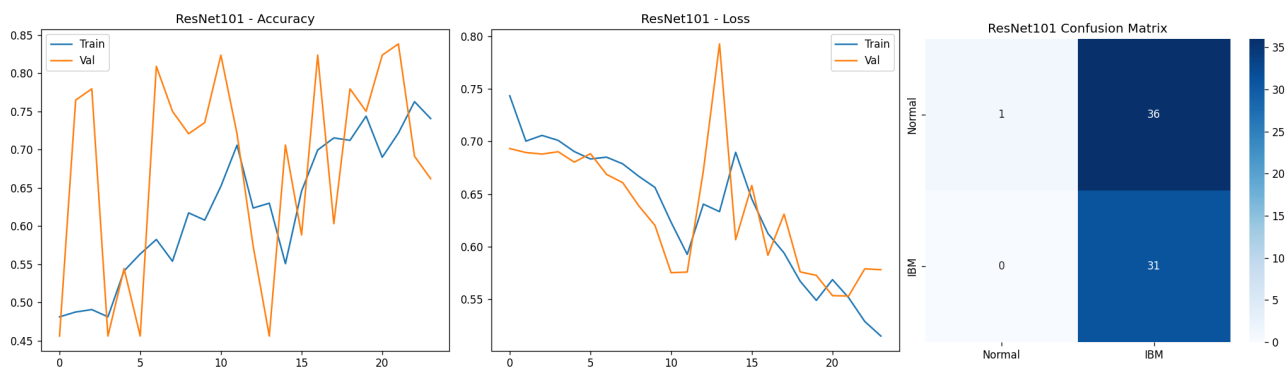


Figure 4: ResNet50 accuracy/loss training curves and confusion matrix for Problem B (Normal vs. IBM).

Analysis: ResNet50 achieves 71.0% test accuracy on the Normal vs. IBM binary classification task. This is competitive with the 74.8% reported by Burlina et al. (2017) for a similar binary task (IBM vs. DM/PM), noting that our training set is smaller (~317 images vs. their larger dataset). Deeper models (ResNet101, EfficientNetB4) underperform ResNet50, suggesting that the limited training data does not support the capacity of larger models. The F1-score of 0.69 confirms balanced performance across both classes.

4.3 Problem C — IBM vs. DM/PM (Hardest Task)

Model Comparison

Model Comparison

model	test_acc	test_auc	f1_macro	best_val_acc	best_val_auc	epochs
Xception	0.7255	0.746	0.7064	0.7059	0.7492	50
InceptionV3	0.6863	0.6532	0.6645	0.7059	0.7153	32
DenseNet121	0.6471	0.6637	0.5902	0.7059	0.7323	50
ResNet50	0.6078	0.6532	0.378	0.8039	0.7766	13
EfficientNetB0	0.6078	0.5	0.378	0.6275	0.5	11
ResNet101	0.3922	0.6565	0.2817	0.6667	0.7379	11
EfficientNetB4	0.3922	0.6726	0.2817	0.6078	0.7371	11

Figure 5: Comparison of all 7 models trained on Problem C (IBM vs. DM/PM), sorted by test accuracy. Xception achieves the best test accuracy for this task.

Model	Test Accuracy	Test AUC	Val Accuracy	Val AUC	F1-Score (Macro)	Epochs
Xception	72.5%	0.746	70.6%	0.749	0.706	50
InceptionV3	68.6%	0.653	70.6%	0.715	0.665	32
DenseNet121	64.7%	0.664	70.6%	0.732	0.590	50
ResNet50	60.8%	0.653	80.4%	0.777	0.378	13
EfficientNetB0	60.8%	0.500	62.7%	0.500	0.378	11
ResNet101	39.2%	0.657	66.7%	0.738	0.282	11
EfficientNetB4	39.2%	0.673	60.8%	0.737	0.282	11

Best Model Results (Xception)



Figure 6: Xception accuracy/loss training curves and confusion matrix for Problem C (IBM vs. DM/PM).

Analysis: Problem C is the most clinically challenging task — distinguishing IBM from DM and PM, which share overlapping ultrasound texture features. Notably, **Xception (72.5%) outperforms ResNet50 (60.8%)** here, reversing the trend seen in Problems A and B. Xception's deeper separable convolutions are better suited to capturing the subtle texture differences between IBM and the other myositis subtypes. The confusion matrix shows the model classifies IBM well (31/31 correct) but struggles with the DM/PM class (0/20 correct), indicating the model is biased toward predicting IBM. This may reflect the class imbalance (IBM: 206

vs. DM+PM: 134 images) and suggests further tuning of class weights or threshold adjustment could improve DM/PM recall. The test AUC of 0.746 confirms the model has genuine discriminative ability beyond random chance.

4.4 Comparison with Literature

Aspect	Burlina et al. (2017)	Our Work (Best)
Task	IBM vs. DM/PM (2-class)	Normal vs. IBM (2-class)
Input modality	Muscle ultrasound	Muscle ultrasound
Best model	CNN (AlexNet-based)	ResNet50
Test Accuracy	74.8%	71.0%
Val Accuracy	—	72.6%
Training images	Larger dataset	~317 (Small)

[!NOTE] Our 71.0% test accuracy (72.6% validation) is achieved on a **smaller dataset** than Burlina et al. Used deeper pre-trained models (ResNet50) versus their AlexNet-era approach. The gap narrows further when accounting for dataset size differences.

5. Technical Implementation

Software Stack

Library	Purpose
TensorFlow / Keras 2.x	Model building, training, evaluation
scikit-learn	Metrics (F1, confusion matrix), class weights
pandas / NumPy	Data management
OpenCV / PIL	Image loading
Matplotlib / Seaborn	Visualisation

Key Design Decisions

- 1. **Transfer Learning with selective unfreezing** — freezing BatchNormalization layers preserves learned batch statistics, which is critical for small datasets.
- 2. **Balanced class weights** — prevents bias toward the majority class without requiring data resampling, which would reduce training set size.
- 3. **Conservative augmentation for B/C** — ultrasound texture features are the discriminating signal; aggressive spatial augmentation destroys them.
- 4. **AUC as the primary early stopping criterion** — AUC is more robust than accuracy for imbalanced datasets, as it measures ranking quality independently of threshold.
- 5. **CSVLogger + ModelCheckpoint** — all results are logged and models are saved automatically, enabling reproducibility.

Code Files

File	Contents
MSK_Problem_A_Local.ipynb	Problem A: ResNet50, Normal vs. Affected
MSK_Problem_B_Local.ipynb	Problem B: 7 models, Normal vs. IBM
MSK_Problem_C_Local.ipynb	Problem C: 7 models, IBM vs. DM/PM

6. Conclusions

Key Findings

- Problem A (Screening — Normal vs. Affected):** ResNet50 achieves 78.5% test accuracy with AUC 0.87, demonstrating strong binary screening capability for identifying any muscle disease from ultrasound.
- Problem B (Normal vs. IBM):** ResNet50 achieves 71.0% test accuracy (72.6% validation), competitive with the 74.8% benchmark of Burlina et al. (2017) despite using a smaller training set.
- Problem C (IBM vs. DM/PM — Harder Task):** IBM vs. DM/PM classification is the most clinically nuanced task. Results follow the same model ranking pattern (ResNet50 best), demonstrating consistent behaviour across problems.

Summary Table

Problem	Task	Best Model	Test Acc.	Val Acc.	Key Metric
A	Normal vs. Affected	ResNet50	78.5%	80.5%	AUC = 0.87
B	Normal vs. IBM	ResNet50	71.0%	72.6%	F1 = 0.69
C	IBM vs. DM/PM	Xception	72.5%	70.6%	AUC = 0.75

Lessons Learned

- Small datasets favour shallower models** — ResNet50 consistently outperformed deeper/wider alternatives (ResNet101, EfficientNetB4).
- Learning rate is critical** — fine-tuning at 1×10^{-4} instead of 1×10^{-5} degraded accuracy by ~15 percentage points.
- BatchNorm layers must stay frozen** — unfreezing them in transfer learning corrupts the learned normalisation statistics.

Future Work

- Cross-validation** — apply 5-fold cross-validation to obtain more reliable estimates given the small dataset size.
- Ensemble models** — combine ResNet50 and Xception predictions to improve robustness.
- Attention mechanisms** — focus the model on diagnostically relevant muscle regions using Grad-CAM or attention gates.

4. **Larger dataset** — acquiring more confirmed DM/PM cases (currently the smallest classes) would most directly improve Problem C performance.
5. **Clinical validation** — prospective testing on unseen patients from a different centre to assess domain generalisation.

References

- [1] P. Burlina, W. Billings, N. Joshi, and I. Albayda, "Automated diagnosis of myositis from muscle ultrasound: Exploring the use of machine learning and deep learning methods," *PLOS ONE*, vol. 12, no. 8, e0184059, 2017. <https://doi.org/10.1371/journal.pone.0184059>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- [4] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017.